Dgrammar and DPGrammar: Frontier Constrained Decoding for Diffusion Language Models

Jeng-Yue Liu[1], Wilson Zheng[1], Haoling Pu[1]
[1]Language Technologies Institute, Carnegie Mellon University
{buffettl,wilsonz,haolingp}@andrew.cmu.edu

Abstract

The leading constrained decoding method for discrete diffusion language models (dLLMs) reports ≈99% validity on simple JSON benchmarks, but this does not transfer to harder schemas. On JSONSchemaBench `medium`, stochastic lookahead-verify achieves only 77.5% validity (n=511), consuming 255.3 lookahead draws per instance and timing out on 73 instances. We address this in two steps. **Dgrammar** replaces stochastic lookahead with deterministic frontier masking, AIMD batching, and async mask overlap, raising validity to 84.9% and eliminating all timeouts. **DPGrammar** adds a Viterbi DP that jointly repairs violated token spans over grammar DFA states, reaching **99.2%** validity with 1.06 mean resamples. Code: https://github.com/buffett0323/anlp_final.

1 Introduction

Many applications require model outputs to satisfy a formal grammar or schema: APIs that consume JSON, tools that compile code, pipelines that validate structured records. Discrete diffusion language models (dLLMs) such as LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025) are attractive because they refine many tokens simultaneously under bidirectional context, but this parallel update breaks the monotonic prefix on which classical Finite-State Machine (FSM)[1] constrained decoding is built. Ensuring schema compliance without destroying the model's joint proposals is therefore an open problem with direct deployment consequences.

The leading constrained decoding method for dLLMs is LAVE (Zhang et al., 2026). While LAVE achieves near-perfect validity on `json-mode-eval` (Nous Research, 2024), our evaluation on the more complex JSONSchemaBench (Geng et al., 2025) `medium` reveals a performance gap, with validity decreasing to 78.9% on the n=511 instances for which LLGUIDANCE accepts the schema (Section 3) due to the benchmark's intricate nested structures and cross-field constraints. The gap is not incidental: LAVE verifies each frontier token by drawing K random completions and accepting only if at least one is grammar-valid. On simple schemas, random suffixes are likely to be valid; on schemas with joint multi-token constraints, the probability that a random suffix satisfies all constraints simultaneously drops sharply, causing false rejections, K-fold forward cost per token, and stochastic run-to-run instability.

Figure 1 concretizes the three regimes above: stochastic lookahead, greedy frontier remasking, and joint span repair on one running instance. We address the root cause in two steps. **Dgrammar** replaces stochastic sampling with deterministic frontier masking: grammar constraints use the incremental `LLMatcher` from LLGUIDANCE (Microsoft, 2025) at the first unresolved position only; committed spans advance the parser state without replaying from scratch; AIMD[2] adaptive batching unmasks a block per forward pass; violators are localized and resampled in-place; and `compute_mask` runs asynchronously overlapping the GPU forward, where no random completions drawn. This engineering redesign raises validity to 84.9% and cuts mean wall time from 39.48 s to 13.14 s (≈3×). When a batch unmask produces tokens with inter-dependent joint violations (e.g., an enum mismatch at c and a regex-pattern violation at $s-1$ must be satisfied simultaneously), greedy per-position repair still falls short. **DPGrammar** adds `dp_fix_prefix`: a Viterbi dynamic program that jointly reassigns the entire violated span $[c, s)$ over grammar DFA states and

[1]FSM refers to the automaton that tracks which tokens are valid at each position given the grammar/schema.

[2]Adapted from the Additive Increase/Multiplicative Decrease (AIMD) principle in TCP congestion control: B increases by 1 upon success and resets to 1 upon a grammar violation.

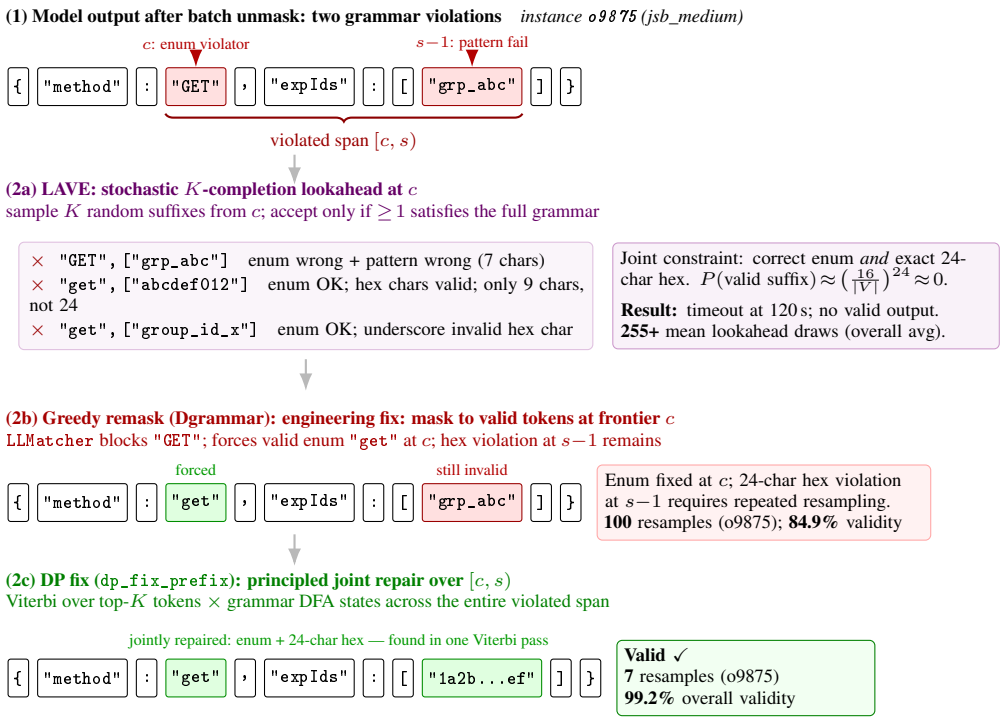

Figure 1: Motivating example from JSONSchemaBench medium instance `o9875` (API route config). The schema requires `method` ∈ `["get","post","put",...]` (lowercase enum) and each `experienceGroupIds` item to match `^[A-Fa-f0-9]{24}$` (24-character hex, e.g. MongoDB ObjectId). **(1)** The diffusion model's batch unmask produces `"GET"` (fails lowercase enum, position c) and `"grp_abc"` (fails 24-char hex pattern, position $s-1$); the violated span covers both. **(2a) LAVE** samples K random completions from c: satisfying both the enum *and* the exact-24 hex pattern simultaneously requires $P \approx (16/|V|)^{24} \approx 0$, no valid completion is ever drawn; LAVE times out at 120 s (255+ mean draws overall). **(2b) Dgrammar** forces `"get"` at c via `LLMatcher` masking (enum fixed), but the hex-pattern violation at $s-1$ requires 100 resamples to generate a valid 24-character string (84.9% overall validity). **(2c) DPGrammar** runs Viterbi over $[c, s)$ jointly, assigning both `"get"` and a valid 24-char hex ID in one pass, only 7 resamples on this instance (99.2% overall validity).

top-K token candidates in one pass, pushing validity to **99.2%** with mean resamples of 1.06.

Our contributions are: (i) identifying and quantifying LAVE's benchmark gap, tracing it to stochastic sampling under complex joint constraints; (ii) a deterministic engineering redesign achieving 84.9% validity; (iii) a principled Viterbi repair layer achieving 99.2%, matching LAVE's claimed number, now on a harder benchmark; and (iv) fine-grained timing instrumentation separating grammar-check from forward-pass cost.

2 Related Work

Discrete diffusion and parallel decoding. DLLMs such as LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025) enable non-autoregressive (AR), any-order text generation through iterative masked refinement. This flexibility introduces the incomplete prefix problem: unlike AR models, where a stable left-to-right prefix always exists for grammar-state tracking, dLLMs must predict tokens surrounded by still-masked or partially denoised positions, making it unclear where the parser frontier lies. Blockwise accelerators (e.g., DFlash (Chen et al., 2026), Spiffy (Agrawal et al., 2026)) reduce sampling steps but remain unconstrained, offering no structural or syntactic guarantees. Our work is orthogonal: we fix the backbone and study how to impose grammar constraints despite the incomplete prefix.

Constrained decoding in AR models. Tooling such as Outlines (Willard and Louf, 2023) and XGrammar (Dong et al., 2024) exploits a monotonic prefix to maintain FSM or parser states across generation steps. These interfaces do not transfer to diffusion because there is no single causal frontier until we define one.

Constrained decoding for diffusion. Mündler et al. (2025) propose the first CFG-constrained dLLM decoder; DINGO (Suresh et al., 2025) uses

dynamic programming for syntactic correctness at higher per-step cost. LAVE (Zhang et al., 2026) introduces a lookahead-then-verify mechanism, sampling K random completions to decide whether to accept each frontier token, and has become the dominant baseline, reporting $\approx 99\%$ syntactic validity on `json-mode-eval` (Nous Research, 2024). We show this number is benchmark-specific: LAVE's stochastic sampling fails to cover joint constraint spaces on harder schemas, dropping to 78.9% on JSONSchemaBench (Geng et al., 2025) `medium` on the $n{=}511$ LLGUIDANCE-compatible instances, and its reliance on random completions risks false rejection of genuinely valid tokens. Plan-style decoders (e.g., PVF (Li et al., 2026)) emphasize structure-aware planning; we stay closer to token-level masking with an explicit incremental matcher.

Sparse transition structures and DP cost. STATIC (Su et al., 2026) shows that sparse DFA transition structures can be flattened into Compressed Sparse Row (CSR) matrices, enabling $O(K)$ coalesced lookups per step, where K is the grammar branch factor, rather than scanning the full vocabulary $|V|$. We adapt this principle to DPGrammar: instead of scoring all $|V|$ tokens at each DP position, we restrict the Viterbi search to top-K candidates per position, reducing per-step transition cost from $O(|V|)$ to $O(K)$ while preserving coverage of the most probable grammar-feasible paths.

3 Data

We evaluate on `jsb_medium`, a JSON Schema benchmark split used in our DLLM evaluation harness, aligned with the JSON schema structured output setting discussed in broader benchmarks (Geng et al., 2025). Each instance provides a natural language style prompt, a schema string, and evaluation extracts structured output from the model generation region.

Scale and splits. We merge JSONL logs from nine Modal shards (seed 0, $T{=}128$, offsets $0, 66, \ldots, 528$) on `instance_id`. The driver originally targets $n{=}586$ instances. Dgrammar and DPGrammar build constraints with LLGUIDANCE; when JSON Schema compilation fails (unsupported keywords, draft incompatibilities, circular references, or malformed specs), no matcher is created and that instance cannot be run. This removes **75** instances. For a controlled comparison, we evaluate **all three methods on the same remaining** $n{=}511$ **instances**, we exclude those 75 cases for LAVE as well, so headline metrics refer to an identical instance list even though LAVE's own codebase can ingest a wider class of schemas.

Preprocessing. Instances without a usable schema string are skipped in the driver. No additional manual annotation is performed; validity is judged by the benchmark's extraction and schema check, surfaced as the boolean `valid` in result JSON lines.

4 Method

4.1 Baselines reproduced

Our primary external baseline is LAVE (Zhang et al., 2026), reimplemented to operate with our LLaDA-8B-Instruct backbone and `jsb_medium` loader under the same diffusion step budget $T{=}128$ where applicable. This isolates the constraint algorithm rather than changing the base model. We additionally compare Dgrammar against DPGrammar on identical instances.

4.2 Dgrammar: proposed deterministic frontier stack

From the engineering perspective, we target two coupled issues in constrained decoding for masked diffusion LLMs: *(i)* hard masking or lookahead over whole spans can distort the model's joint proposals across positions; *(ii)* per-step grammar checks and mask construction can add latency on top of already expensive diffusion forwards. We avoid stochastic suffix sampling entirely and instead enforce the schema only at the decoding frontier, while reusing parser state and hiding much of the constraint work behind the next GPU forward.

Incremental grammar backbone. We build on LLGUIDANCE (Microsoft, 2025), which exposes an incremental finite-state view of the schema: each newly committed token advances a persistent automaton state, so we never re-scan the full prefix from the prompt after every block update.

Frontier-only constraints. Constraint enforcement applies at the first position that is still undecided. We derive the allowed next-token set there from the current automaton state; elsewhere, positions may remain masked while we read committed text left to right. Logits stay unconstrained off the

frontier, and we avoid locking down an entire parallel row before the batch of unmask decisions is mutually consistent.

State across denoising steps. As spans stabilize across diffusion iterations, we carry the automaton forward rather than re-initializing from the prompt on every check, which keeps repeated validity tests cheap.

Adaptive commit (AIMD). When consumption succeeds we grow how many tokens we commit per step, up to $k_{\max}=8$; when a conflict appears we shrink the batch, locate the first offending position, repair it, and only then widen the commit window again.

Violator remask and in-place resampling. We mask only the offending token, set its logit to $-\infty$ in the current forward output, and walk successive argmax alternatives on that same tensor so many repairs do not require another GPU forward.

Early termination and asynchronous overlap. We halt once the automaton indicates a complete, schema-valid string, skipping superfluous late diffusion steps. Separately, we prepare the next frontier's allowed-token mask on a CPU worker while the GPU executes the following forward, so mask construction overlaps forward time and reduces wall-clock constraint overhead.

4.3 DPGrammar: segment DP on top of Dgrammar

DPGrammar reuses the same denoising and frontier schedule as Dgrammar; the only difference is how we repair a rejected continuation. Greedy single-position remasking can leave correlated errors when a batch unmask reveals several dependent tokens at once. DPGrammar therefore treats the violated interval $[c, s)$ (from the first offending position up to the next still-masked boundary) as a single joint decision. We run a Viterbi-style dynamic program over that segment, combining grammar automaton states with the $K=100$ highest-scoring vocabulary entries at each position under the current forward (other settings may use a different K). The DP searches for a replacement sequence that restores a consistent parse; if no feasible path exists within that top-K beam, we fall back to Dgrammar's greedy remask. Figure 2 sketches the shared stack and where this segment-level repair attaches. The full pseudocode is given in Appendix A.

4.4 Completion after generation

Both systems share driver side repair after the diffusion phase: grammar guided greedy autocomplete, masked extension, grammar only closing, and minimal JSON synthesis from the schema when required.

4.5 Implementation and compute

Code builds on LLGUIDANCE (Microsoft, 2025), and the constrained diffusion CD4dLLM style bindings (Mündler et al., 2025; Zhang et al., 2026); benchmarks run on Modal with 9× NVIDIA A100 GPUs in our experiments. Backbone is GSAI ML LLaDA-8B-Instruct[3]. Block length 32 for timed async runs, remasking low confidence, temperature 0.2 in the driver. Full hyperparameter are shown in appendix.

4.6 Experimental design and metrics

Schema validity is the primary outcome (binary `valid` per instance). **Wall time** per instance measures end-to-end driver time; Table 1 emphasizes **mean** seconds per instance alongside validity and resample counts. **Constraint overhead %** is driver-reported grammar and mask work as a fraction of total time; **effective constraint %** further attributes mask-wait and autocomplete to constraint cost for Dgrammar/DPGrammar. **Resamples** counts remasking (for LAVE, lookahead verification draws). Per-token timings average constraint and total work over decoded length.

5 Results

Table 1 reports aggregate metrics on the $n=511$ instances retained after excluding schemas that LLGUIDANCE cannot compile (Section 3). We discuss results in the two-step framing of our approach.

LAVE vs. Dgrammar (engineering step). On this matched set, LAVE reaches 78.9% validity (403/511) with 255.32 mean lookahead draws per instance and 39.48 s mean wall time. Dgrammar replaces stochastic sampling with deterministic frontier masking: validity rises to 84.9% (434/511), mean wall time falls to 13.14 s ($\approx 3\times$ faster than LAVE), and mean resamples fall to 32.43. The engineering redesign—incremental parser state, AIMD batching, async overlap—accounts for this gain without a new theoretical primitive.

[3]https://huggingface.co/GSAI-ML/LLaDA-8B-Instruct

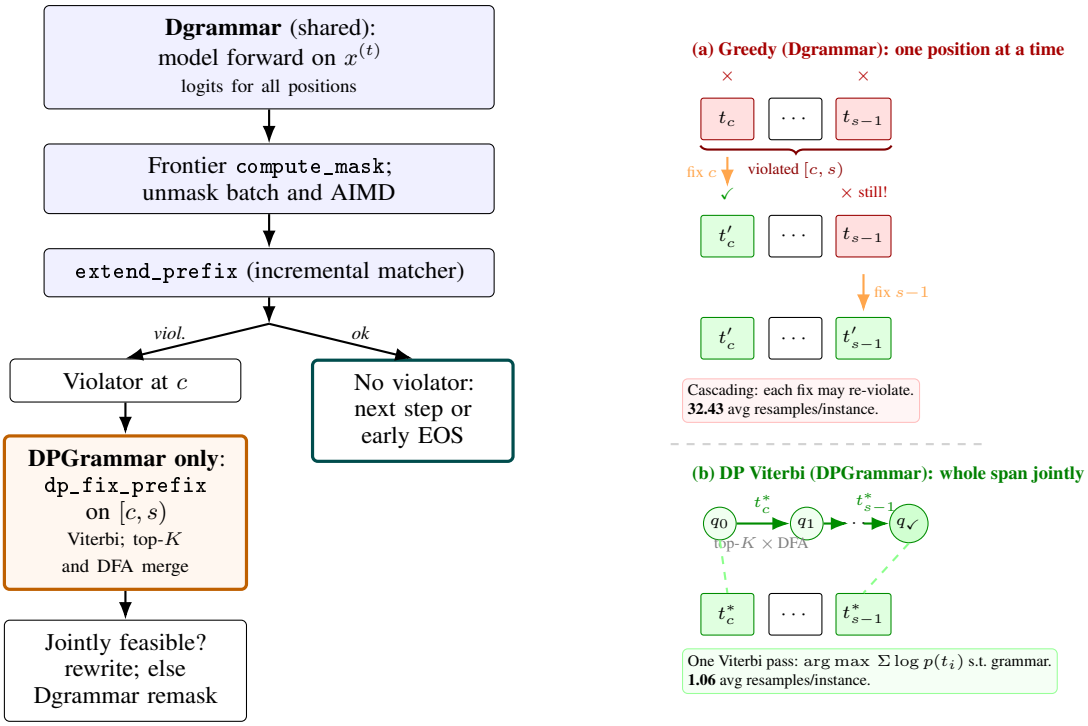

Figure 2: **Left**: one denoising step under Dgrammar (blue) and the DPGrammar branch (orange). **Right**: why joint repair matters. **(a)** Greedy fixes one position at a time: after forcing the correct token at c, the constraint at $s-1$ may still be violated, triggering another resample, which can itself cascade, 32.43 mean resamples per instance. **(b)** The Viterbi DP sweeps the entire span $[c, s)$ once, tracking grammar DFA states and selecting the highest-log-prob jointly grammar-valid token sequence, 1.06 mean resamples per instance.

Dgrammar vs. DPGrammar (principled step). Dgrammar's residual failures stem from greedy per-position repair that misses joint constraint structure (e.g., fixing a key but leaving an incompatible value type in the same span). DPGrammar adds `dp_fix_prefix`: a Viterbi search over the full violated span under grammar DFA states. Validity rises to 99.2% (507/511), mean resamples collapse from 32.43 to 1.06, while mean wall time moves from 13.14 s to 14.96 s—a modest increase relative to the validity gain. Effective constraint time as a fraction of wall clock drops sharply (Table 1). The principled extension closes what the engineering step cannot: multi-token joint violations that greedy remasking iterates through slowly.

Qualitative observations. Error modes for the remaining 0.8% DPGrammar failures include completion-phase limits and schemas where minimal-JSON fallback is brittle. Detailed case studies are left to appendix space.

6 Discussion

The LAVE–Dgrammar gap shows that deterministic frontier parsing is a strong engineering solution: it eliminates sampling variance and timeout risk with no new theoretical machinery. The Dgrammar–DPGrammar gap, in turn, shows that greedy token-by-token repair has a structural limit, joint violations spanning multiple positions call for a principled combinatorial search, not more engineering tuning. The two steps are complementary: the engineering stack is prerequisite infrastructure; the DP layer addresses what greedy repair leaves unresolved. Empirically, DPGrammar attains the highest validity and the lowest resample churn, while Dgrammar remains an attractive option wherever the overhead of segment DP is unwelcome.

These results should be read with several caveats. Effective constraint % and mask timings are not equally interpretable across methods (LAVE lacks the same breakdown). DPGrammar's mean wall time is slightly above Dgrammar's despite far fewer resamples, reflecting DP work on violation spans. DPGrammar still falls short of 100% validity, so driver-side completion and fallback paths remain load-bearing rather than cosmetic.

Finally, practitioners should report forward versus constraint-time splits, not only end-to-end latency. For the research community, joint repair over

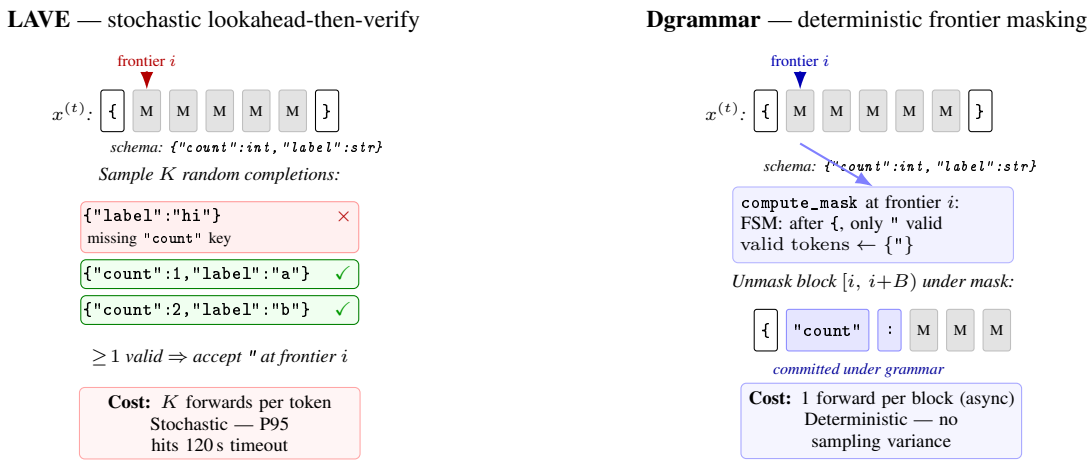

Figure 3: One denoising step under schema `{"count":int,"label":str}`. **LAVE** (Zhang et al., 2026) (left) samples K random completions at each frontier token, accepting it only when ≥ 1 suffix is valid—stochastic and $K\times$ more expensive per token, hitting the 120 s per-instance timeout on hard schemas. **Dgrammar** (right) runs `compute_mask` on the incremental FSM state and unmasks an entire block under that mask in one forward pass, overlapped with the next mask computation; the result is deterministic.

Metric	**LAVE** (Zhang et al., 2026)	**Dgrammar** (ours)	**DPGrammar** (ours)
Skipped (grammar-invalid)	75	75	75
Evaluated (n)	511	511	511
Valid	396 / 511	434 / 511	**507 / 511**
Validity (%)	77.5%	84.9%	**99.2%**
Timeouts (>120 s)	73	0	0
Mean resamples [§]	255.3	32.43	**1.06**
Mean time (s)	27.4	24.2	**15.0**
Median time (s)	21.9	20.6	**14.5**
P95 time (s)	73.4	51.7	**29.5**
Max time (s)	113.2	96.7	**62.5**

Table 1: `jsb_medium` results (seed 0, T=128 steps). All three methods are evaluated on the same n=511 matched instances; 75 instances are skipped by all methods due to grammar-invalid schemas (LAVE: immediate parse error; Dgrammar/DPGrammar: silently skipped by LLGUIDANCE). [§] LAVE resamples count lookahead verification draws; Dgrammar/DPGrammar resamples count token re-mask events.

short violation spans is a comparatively lightweight addition on top of a frontier decoder, relative to full-sequence dynamic programs (Suresh et al., 2025).

7 Conclusion and Future Work

We studied grammar constrained decoding for dLLMs in two steps on n=511 LLGUIDANCE-compatible instances. Dgrammar is an engineering redesign of the constraint loop (deterministic frontier masking, incremental parser state, AIMD batching, async overlap) that raises validity from 78.9% (LAVE) to 84.9% and cuts mean wall time from 39.48 s to 13.14 s without any new theoretical primitive. DPGrammar is the principled extension: Viterbi-based joint repair over violated spans pushes validity to 99.2% and collapses resamples from 32.43 to 1.06, addressing multi-token joint constraints that greedy repair cannot resolve efficiently. Broader takeaway: the engineering step provides the infrastructure; the principled step provides the correctness guarantee; both are needed for near-perfect structured generation under diffusion.

Future work includes extending to additional JSON splits or code grammars; and exploring tighter integration with sparse automata acceleration (Su et al., 2026) for very large vocabularies.

8 Limitations

Table 1 is restricted to n=511 instances: we exclude 75 benchmark cases whose JSON Schema LLGUIDANCE rejects at compile time, and we evaluate LAVE on that same filtered list so that all columns are instance-aligned. A separate LAVE-only run on the full 586-instance merge would in-

clude those 75 (typically `valid=false`); we do not claim Table 1 summarizes the full unfiltered benchmark. The validity flag depends on benchmark extraction; functional tests beyond schema syntax are not central here. Hyperparameter tuning was limited; different T, block sizes, or temperature could shift outcomes. DPGrammar explores top K tokens per DP position, which can miss globally optimal tokens outside the top K.

9 Ethical Considerations

Structured generation can reduce malformed outputs in downstream automation, but valid JSON is not always semantically safe or fair: schemas can encode biased or incomplete coverage of user intent. Models and grammars should be reviewed for misuse in high stakes settings (medical, financial). Our benchmark instances originate from public style schema tasks; they may not represent all languages or cultures. Researchers and deployers should combine syntactic guarantees with content moderation and policy controls. We use open backbone weights and public style evaluation data; users of our code should respect model licenses and avoid deploying unreviewed systems in sensitive domains without human oversight.

References

Sudhanshu Agrawal, Risheek Garrepalli, Raghavv Goel, Mingu Lee, Christopher Lott, and Fatih Porikli. 2026. Spiffy: Multiplying diffusion llm acceleration via lossless speculative decoding. *Preprint*, arXiv:2509.18085.

Jian Chen, Yesheng Liang, and Zhijian Liu. 2026. Dflash: Block diffusion for flash speculative decoding. *Preprint*, arXiv:2602.06036.

Yixin Dong, Charlie F Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. Xgrammar: Flexible and efficient structured generation engine for large language models. *Proceedings of Machine Learning and Systems 7*.

Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. 2025. Jsonschemabench: A rigorous benchmark of structured outputs for language models. *Preprint*, arXiv:2501.10868.

Miao Li, Hanyang Jiang, Sikai Cheng, Hengyu Fu, Yuhang Cai, Baihe Huang, Tinghan Ye, Xuanzhou Chen, and Pascal Van Hentenryck. 2026. Plan, verify and fill: A structured parallel decoding approach for diffusion language models. *Preprint*, arXiv:2601.12247.

Microsoft. 2025. LLGuidance: Fast structured outputs for LLMs. GitHub repository.

Niels Mündler, Jasper Dekoninck, and Martin Vechev. 2025. Constrained decoding of diffusion llms with context-free grammars. *Preprint*, arXiv:2508.10111.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large language diffusion models. *Preprint*, arXiv:2502.09992.

Nous Research. 2024. json-mode-eval. Hugging Face Datasets.

Zhengyang Su, Isay Katsman, Yueqi Wang, Ruining He, Lukasz Heldt, Raghunandan Keshavan, Shao-Chuan Wang, Xinyang Yi, Mingyan Gao, Onkar Dalal, Lichan Hong, Ed Chi, and Ningren Han. 2026. Vectorizing the trie: Efficient constrained decoding for llm-based generative retrieval on accelerators. *Preprint*, arXiv:2602.22647.

Tarun Suresh, Debangshu Banerjee, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. 2025. Dingo: Constrained inference for diffusion llms. *Preprint*, arXiv:2505.23061.

Brandon T Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7b: Diffusion large language models. *Preprint*, arXiv:2508.15487.

Yitong Zhang, Yongmin Li, Yuetong Liu, Jia Li, Xiaoran Jia, Zherui Li, and Ge Li. 2026. Lookahead-then-verify: Reliable constrained decoding for diffusion llms under context-free grammars. *Preprint*, arXiv:2602.00612.

A DPGrammar Algorithm (`dp_fix_prefix`)

Algorithm **??** gives the full pseudocode for the Viterbi segment repair used by DPGrammar. Given the violated span $[c, s)$, the algorithm maintains a set of live DFA states (each paired with its highest cumulative log-probability path) and advances them token by token using the top-K vocabulary entries from the current forward pass. When two paths arrive at the same DFA state, the lower-scoring one is pruned (Viterbi merge). After processing all positions, the algorithm backtracks from the best final state to recover the replacement token sequence. Matcher clones are created only once per surviving DFA state per step (Phase 2), keeping memory overhead proportional to the grammar's state count rather than to K.

B Hyperparameters

All experiments use LLaDA-8B-Instruct as the backbone, run on Modal with NVIDIA A100 GPUs. Table 2 lists the shared settings across all three methods.

Hyperparameter	Value
Diffusion steps T	128
Random seed	0
Block length	32
Remasking strategy	Low-confidence
Temperature	0.2
Top-K (DPGrammar DP)	50
Max resample attempts	500 (LAVE), 200 (Dgrammar/DPGrammar)
Timeout threshold	120 s

Table 2: Shared hyperparameters for all benchmark runs on `jsb_medium`.

C Dataset and Instance Filtering

We evaluate on the `Github_medium` split of JSONSchemaBench, which contains 586 instances. Of these, 75 are excluded from all comparisons because LLGUIDANCE rejects their schemas at compile time: LAVE immediately returns `valid=false` with the error `"Prompt does not conform to grammar"`; Dgrammar and DPGrammar silently skip them via the same LLGUIDANCE guard. Since all three methods behave identically on these 75 instances, including them would only inflate LAVE's failure count without providing a meaningful comparison. All results in Table 1 are therefore reported on the matched n=511 subset.

D Timing Breakdown

Table 3 reports additional per-method timing statistics not included in the main table. Forward time dominates in all cases; constraint overhead is a small fraction for Dgrammar and DPGrammar. LAVE's higher mean time is driven by the 73 timeout instances (capped at 120 s); its median is lower because the majority of instances either succeed quickly or fail early.

Metric	LAVE	Dgrammar	DPGrammar
Avg fwd passes	99.2	104.7	99.4
Constraint overhead (%)	11.95	24.25	12.67
Eff. constraint (%)	—	20.43	1.22
Per-token total (ms)	111.5	100.7	71.9
Per-token constraint (ms)	6.64	8.26	0.30
Avg mask compute (ms)	—	427.2	577.0
Avg forward total (ms)	3951	4612	4229

Table 3: Detailed timing breakdown on `jsb_medium` (n=511 matched instances). Effective constraint % is not available for LAVE as it lacks the same instrumentation.